SELF-HEALING MLOps PLATFORM

B.TECH FINAL YEAR PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY (B.TECH)
IN
COMPUTER SCIENCE & ENGINEERING

by

Maimoona Manzoor (CSE-22-LE-70)
Syed Maryam Andrabi (CSE-22-LE-74)
Momin Zahoor (CSE-22-LE-71)

Guided by

Dr Sahil Sholla
Assistant Professor, Department of CSE, IUST

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ISLAMIC UNIVERSITY OF SCIENCE AND TECHNOLOGY
AWANTIPORA, JAMMU & KASHMIR, INDIA - 192122
Year: 2026

CERTIFICATE

This is to certify that the project report entitled "**SELF-HEALING MLOps PLATFORM**" submitted by **Maimoona Manzoor** (CSE-22-LE-70), **Syed Maryam Andrabi** (CSE-22-LE-74), and **Momin Zahoor** (CSE-22-LE-71) to the Department of Computer Science & Engineering, Islamic University of Science and Technology, Awantipora, is a record of bonafide work carried out by them under my supervision and guidance in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering.

The work embodied in this report has been carried out at the Department of Computer Science & Engineering, IUST, and has not been submitted elsewhere for the award of any other degree or diploma.

Dr. Sahil Sholla (Project Guide)	Head of Department, CSE

DECLARATION

We hereby declare that the project report entitled "SELF-HEALING MLOPS PLATFORM" submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at the Islamic University of Science and Technology, Awantipora, is an authentic record of our own work carried out during the academic year 2025-2026 under the supervision of Dr. Sahil Sholla, Assistant Professor, Department of CSE.

We further declare that the work reported in this project report has not been submitted, either in part or in full, to any other university or institution for the award of any other degree or diploma.

Maimoona Manzoor (CSE-22-LE-70)

Syed Maryam Andrabi (CSE-22-LE-74)

Momin Zahoor (CSE-22-LE-71)

ACKNOWLEDGEMENT

We express our sincere and heartfelt gratitude to our project guide, Dr. Sahil Sholla, Assistant Professor, Department of Computer Science & Engineering, for his continuous support, invaluable guidance, and constant encouragement throughout every stage of the design, development, and documentation of this project. His technical insight and constructive feedback were instrumental in shaping the direction and quality of this work.

We are also thankful to the Head of the Department and the faculty members of the Department of Computer Science & Engineering, Islamic University of Science and Technology, Awantipora, for providing the necessary infrastructure, laboratory facilities, and an academic environment conducive to carrying out this project.

Finally, we would like to thank our families and peers for their unwavering support and encouragement throughout the duration of this project.

ABSTRACT

The Self-Healing MLOps Platform is a system that helps to automate the entire process of machine learning. This includes getting data preparing it training models putting them to use keeping an eye on them and training them again when needed. The Self-Healing MLOps Platform does all this with little help from people. It uses workflows and checks for changes in the data to manage the models all the time. The system uses models like Random Forest, XGBoost and LightGBM. It also uses Optuna to find the settings for these models MLflow to track experiments and Evidently AI to detect changes in the data. The backend of the system is built with FastAPI. It provides services to make predictions. The dashboard is built with React. It shows how the models are doing the health of the system and if there are any changes in the data in real time. The system uses Docker to keep everything contained. The Self-Healing MLOps Platform has been tested using the IBM HR Employee Attrition dataset. It is designed to work with datasets that are organized in tables. It does not need much setup. By putting automation, monitoring and self-healing all together the Self-Healing MLOps Platform shows how to build machine learning applications that're reliable and can handle a lot of work. This is what modern MLOps practices are, about. The Self-Healing MLOps Platform is an example of this.

Keywords: MLOps, Self-Healing Systems, Drift Detection, Automated Retraining

Table of Contents

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION

1.1 Background

The growth of Artificial Intelligence and Machine Learning in businesses has created a problem. This problem is that managing Machine Learning systems is mostly done by hand. A lot of work has gone into making models and training them. But the systems needed to put these models into action and keep them running are still mostly manual. This means they are not very strong and cannot handle a lot of work.

Traditional Machine Learning operations set-ups need people to do a lot of work. This work includes getting data ready training models watching them and training them again. Also Machine Learning models tend to get less accurate after they are put into action. This happens because the data they are using is changing all the time. This is called model drift. It makes the models not work as well. Research shows that people who work with Machine Learning spend most of their time doing maintenance work. They spend their time getting models ready writing scripts watching to see if the models are working, finding problems and training the models again from the start. They do not have a lot of time to work on making models. This is not a waste of time it also means that models are not consistent and can have mistakes.

The problem is even bigger because of something called data drift. The data that Machine Learning models use is always changing. Peoples behavior is changing the market is. The data that is coming in is different from the data that the model was trained on. This means that the model gets less accurate over time. It does not send out any error messages so it is hard to notice. Most organizations do not have a way to find and fix this problem.

Big companies like Uber, Airbnb, Netflix, Facebook and LinkedIn have seen this problem and have solved it. They have built their Machine Learning platforms. These platforms are not available, to everyone. They are closed. It took a lot of work to build them. This means that smaller teams and schools cannot use them. The Self-Healing Machine Learning operations Platform is trying to solve this problem. It wants to make

a platform that's open and easy to use. Any team should be able to use it without having to do a lot of work to set it up.

1.2 Problem Statement

Machine learning models that are used in real-world applications need work. This includes preparing data training the models putting them into use watching how they perform and updating them when needed. These steps are usually done by people. This makes the whole process take a time have more mistakes and be hard to grow. The models might not work well over time because the data changes. Also the tools that companies use are costly. Make it hard to switch to another system. Open-source tools help with parts of the process. Don't cover everything. So there is a need for a automatic system for machine learning operations. This system should keep track of how models are doing, spot when data changes and fix problems on its own with little help, from people.

1.3 Objectives

The project pursues four objectives, each addressing a distinct functional domain of the platform:

Objective 1: To develop an automated machine learning pipeline for data preprocessing, multi-model training and intelligent model selection with minimal human intervention.

Objective 2: To implement continuous data drift monitoring and autonomous model retraining, ensuring reliable and up to date model performance in production.

Objective 3: To design an interactive dashboard for real time performance, training metrics, drift status and system health.

Objective 4: To implement secure data handling and API authentication to ensure integrity across the ML lifecycle.

1.4 Scope of the project

The platform is validated using the IBM HR Employee Attrition dataset, a publicly available, structured, tabular dataset. Since the platform is architected as a domain-agnostic system, it is designed to support any structured tabular dataset with minimal configuration changes, and the HR Attrition dataset serves purely as a reference demonstration dataset rather than a constraint on the platform's applicability.

The current scope of the project is limited to classification problems on structured tabular data. Deep learning pipelines for unstructured data such as images, text, or time-series signals, as well as multi-tenant deployment and federated learning, are treated as future extensions and are discussed in Chapter 6.

1.5 Organisation of the report

The remainder of this report is organised into five further chapters, followed by references and an appendix, as follows:

- Chapter 2 (Literature Review) reviews existing academic research and industry work relevant to MLOps, drift detection, automated pipelines and identifies the gap addressed by this project.
- Chapter 3 (Methodology / System Design) describes the proposed system, the hardware and software requirements, the system architecture, the process flow of the self-healing pipeline, and the technology framework.
- Chapter 4 (Implementation) describes the engineering implementation of each functional module of the platform.
- Chapter 5 (Results and Discussion) presents the results and screenshots obtained from the working system, discusses module-wise completion status, and reflects on the challenges encountered during development.
- Chapter 6 (Conclusion and Future Scope) summaries the engineering contribution of the project and outlines directions for future work.
- The References section lists all academic papers and technical documentation cited in this report, followed by an Appendix summarizing the technology stack and configuration used in the platform.

CHAPTER 2

LITERATURE REVIEW

This literature review was conducted to provide a strong foundation for the present study by examining four key areas relevant to the proposed research.These areas are hidden technical debt in machine learning, software engineering for automated pipelines, concept and data drift detection, and production machine learning in MLOps platforms. By reviewing existing research in these domains, this chapter highlights current developments, identifies limitations in existing approaches, and establishes the research gap. These findings provide the rationale for the proposed Self-Healing MLOps Platform and demonstrate how it aims to address challenges that remain insufficiently explored in the current literature.

2.1 Hidden Technical Dept in Machine learning Systems

Sculley et al. [1] have conducted a popular study, which revealed the existence of a so-called hidden technical debt in production ML systems. The authors showed that the majority of code in a production ML system is not model code, but code related to data pipelines, serving infrastructure, monitoring, and configuration management. This article laid the foundation for the research on the operational-automation problem that the Self-Healing MLOps Platform is designed to solve. The study proved that operations costs increase faster than linearly with system complexity if ML infrastructure is not engineered properly.

2.2 Software Engineering for Machine Learning

Amershi et al. [2] performed an extensive case study at Microsoft on software engineering practices for machine learning and found that model deployment and post-deployment monitoring dominates the time spent on engineering tasks related to ML, with operational automation and monitoring presenting the greatest challenges in production ML. This served as one of the motivations for the current project – the automated deployment and monitoring architecture.

2.3 Concept Drift Detection and Adaptation

Gama et al. provide an extensive overview of concept drift detection and adaptation in classification and regression [3]. The authors develop an informative taxonomy of drift detection methods, classifying them into error-rate-based, distribution-based, and ensemble-based approaches, thus laying the theoretical foundation for the statistical drift detection procedure. It is noteworthy that the authors discuss the Kolmogorov-Smirnov test and Population Stability Index as particular cases of distribution-based drift detection, which are implemented in the drift detection module of the Evidently AI software used for this project's analysis.

2.4 Production Machine Learning

Shankar et al. [4] interviewed practitioners in the field about the ways that machine learning solutions are operated within leading technology companies. The study revealed that while automated retraining pipelines and drift monitoring are the most effective investments that a company can make in ML solutions, they are not implemented in the vast majority of production solutions outside of large technology companies. This research highlights the need for the current project, as it addresses a gap in practice between industry standards and the reality of the majority of companies.

2.5 Existing Platforms and the Identified Gap

Cloud basedML pipeline automation service Amazon SageMaker (AWS), GoogleVertex AIs(GCP)andMicrosoft AzureML(Azure)all give vary degrees of end-to-end automation in production. The catch here of course is that cloud infrastructure costs money, you typically get tightly coupled into proprietary, cloud specific vendor ecosystems giving a high vendor lock in, so out of budget for small teams, start-ups or academics. Further assistance is to be found with Brecket al. [5]who provided a "ML Test Score" for production readiness, and Hapke and Nelson [6]who describe concrete patterns and pragmatic approaches to developing the entire ML pipeline.

Each aspect of ML lifecycle is currently covered by an open-source tool individually - MLflow for experiment tracking, EvidentlyAI for drift detection, Apache Airflow for Orchestration, FastAPI for Model Serving etc., but no open-source solution is yet

available as a single, cohesive, self-healing and deployable platform to manage them all together. The Self-Healing MLOps Platform is built exactly to address this need.

Table 2.1 Literature Survey Summary

S.no	Author and year	Area	Contribution
1	Kreuzberger et al. (2022)	MLOps Architecture	Analysed MLOps principles, workflows, and challenges, providing a structured understanding of ML lifecycle automation.
2	Testi et al. (2022)	MLOps Methodology	Proposed an MLOps taxonomy covering development, deployment, monitoring, and maintenance phases. Highlighted key components required for reliable ML operations.
3	Miñón et al. (2022)	Cloud-Based ML Deployment	Developed a framework for automated infrastructure generation and ML pipeline deployment. Supported deployment across cloud and edge environments.
4	Moreschini et al. (2023**)**	ML Pipeline Automation	Presented an end-to-end MLOps pipeline implementation for automating ML workflows. Demonstrated practical integration of ML development and operations.
5	Wazir et al. (2023)	MLOps Framework Analysis	Reviewed existing MLOps approaches and tools. Identified challenges related to automation, scalability, and production deployment.
6	Myakala et al. (2024)	Drift Detection	Introduced an automated concept drift detection and retraining approach. Improved model reliability under changing data conditions.
7	Mittamidi (2024)	Self-Healing ML Systems	Proposed an autonomous pipeline for failure detection and automated recovery. Reduced manual intervention in ML operations.
8	Park et al. (2025)	Automated Model Retraining	Developed a data-driven retraining optimization approach for ML pipelines. Improved continuous model adaptation using updated data.

9	Tanna (2025)	Self-Healing MLOps	Explored automated drift detection and remediation mechanisms. Focused on building resilient ML pipelines with minimal human involvement.
10	Katalay et al. (2025)	Retraining	Proposed an automated retraining pipeline. Addressed model adaptation after distribution shifts.
11	Begum et al. (2026)	Feature Drift Monitoring	Investigated advanced drift detection techniques for ML feature monitoring. Improved reliability of production ML systems.
12	Wong et al. (2026)	Adaptive Retraining Pipeline	Proposed a drift-aware retraining framework with data quality validation. Enabled automated model adaptation in changing environments.
13	El-Hajj and Zeineddine (2026)	Federated MLOps	Combined federated learning with real-time drift detection. Enhanced privacy and scalability for distributed ML systems.

CHAPTER 3

METHODOLOGY

The engineering process was based on a phased iterative development process according to MLOps standards where development of the system was logically divided into 3 subsequent stages over a duration of 12 weeks (e.g., Phase 1: DataSet Prepare & pre-processing, Phase 2: Auto-ML Training, Experiment Tracking, Secure API serve & Drift-Based Self-healing Re-training, Phase 3: Containerization and interactive dashboard) where each stage delivered an incrementally a functional tier of our proposed system. This paper discusses our proposed system, required system analysis, systemarchitecture, flow diagram of our proposed system and technology stack.

3.1 Proposed System

This is a unified platform that will completely automate an entire machine learning lifecycle such that not a single touch point is needed in terms of user involvement starting from data ingesting until self-healing in production. A user uploads a raw tabular data, the platform at once cleans and preprocesses the data, trains multiple candidate models parallely and picks the best performance model to deploy, deploys this best performing model via a secured API to provide predictions, auto-detects statistical drift at production through continuously monitoring incoming data in prod and re-train the model if drift is encountered.

3.2 Requirement Analysis

3.2.1 Hardware Requirements

The following table presents the hardware requirements necessary for the development, testing, and deployment of the Self-Healing MLOps Platform.

Table 3.1: Hardware Requirements

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 (8th Gen or higher)
RAM	8 GB minimum; 16 GB recommended
Storage	50 GB free disk space (SSD preferred)
Internet Connection	A stable internet connection is required for the proper functioning of the system.

3.2.2 Software Requirements

Table 3.2: Software Requirements

Category	Technology / Tool	Version
Operating System	Windows 10/11 or Ubuntu 20.04+	Latest
Programming Language	Python	3.11
ML Libraries	Scikit-learn, XGBoost, LightGBM	Latest stable
Hyperparameter Tuning	Optuna	Latest stable
Experiment Tracking	MLflow	2.x
Drift Detection	Evidently AI	Latest stable
API Framework	FastAPI + Uvicorn	Latest stable
Frontend	React + Vite + Tailwind CSS	Latest stable
Database	SQLite + SQLAlchemy	Latest stable
Containerisation	Docker + Docker Compose	Latest stable
Version Control	Git + GitHub	Latest stable
IDE	VS Code / PyCharm	Latest stable

3.3 System Architechture

The architecture of the platform follows an independent 4-layered module approach. Each module is capable of being deployed on their own as a Docker container and can interact with its adjacent layer via carefully designed APIs and events to maintain loose coupling, scalability and to ensure fault isolation.

Layer 1: Data Ingestion and Preprocessing layer

Input Data raw dataset is loaded using front-end and passed to back-end ML algorithm processing engine.preprocess ingengine- conducts autonomous EDA analysis, missing value imputation, outlier removal/treatment, predictive feature extraction; deals with class imbalance with SMOTE; outputs a final clean dataset, to be used for training, ready to be consumed without any sort of manual data manipulation by the engineering team.

Layer 2: AutoML Training and Experiment Tracking Layer

The dataset that has been processed is sent to the AutoML training engine, which trains Random Forest, XGBoost, and LightGBM models with hyperparameter optimization based on Optuna at the same time. The parameters, metrics, and models for all the training trials are saved automatically in the MLflow system, and the best model is chosen based on a combined evaluation metric and registered in the MLflow Model Registry as the winner to be used for the deployment.

Layer 3: Serving, Security, and Drift Monitoring Layer

The top-performing model has been deployed through a FastAPI REST endpoint protected by JWT authentication and Bcrypt password hashing. All incoming calls for predictions are recorded in an SQLite database using SQLAlchemy. Evidently AI checks the statistical features of the incoming prediction data against the original training data, looking out for drift scores that exceed prescribed thresholds according to the Kolmogorov-Smirnov test and the Population Stability Index.

Layer 4: Dashboard: Presentation Layer

Finally, we've built a complete multi service platform that's containerised with Docker and orchestrated with Docker compose.

The Dashboard acts as the main interface of the Self-Healing MLOps Platform, giving users a clear and organized view of the entire system. It allows users to monitor model

performance, check for data drift and security alerts, and view the status of deployed models in real time. With an easy-to-use and interactive design, the dashboard helps users track system activities and manage the MLOps workflow more efficiently. This concludes our last module.

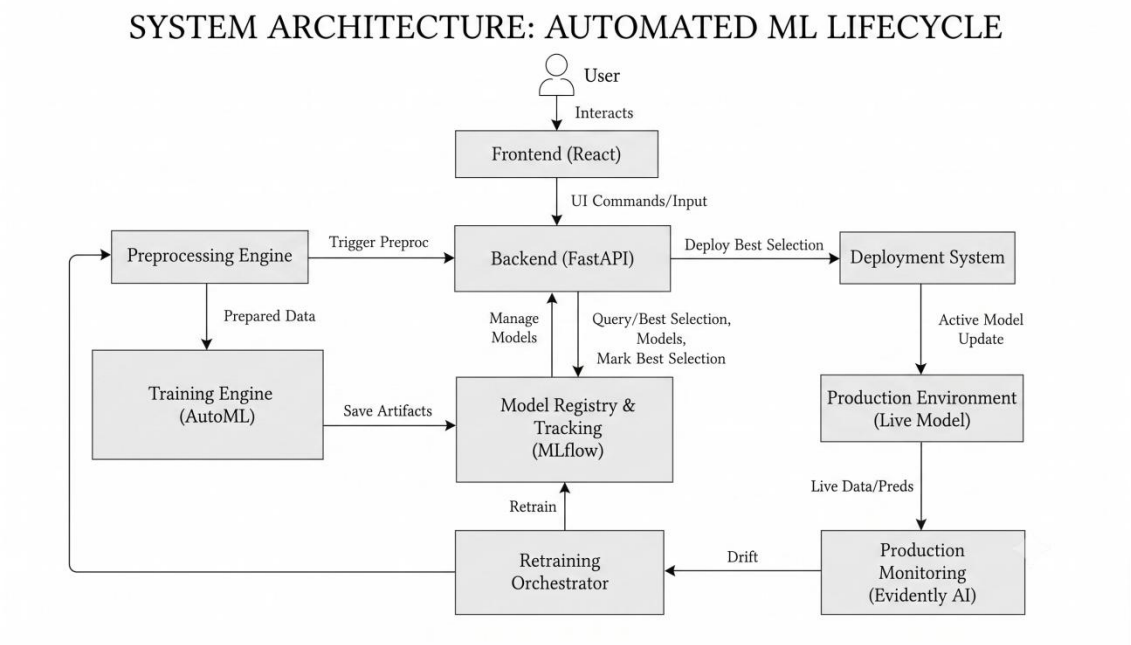

Figure 3.1: System Architecture of the Self-Healing MLOps Platform

3.4 Process Flow of the Self-Healing Pipeline

A general diagram illustrating how this end-to-end workflow works can be seen below in Figure 3.2. When an initial dataset has been uploaded, our system will then run automated cleaning/preprocessing steps on that data set, then use that dataset to simultaneously train our three candidates for the best performing model.

When the best candidate has been identified, it will be deployed and start serving predictions. Meanwhile, Evidently AI will begin constantly tracking both the production data and predictions. If there is no evidence of any kind of drift, Evidently will continue doing this indefinitely, but once it identifies some sort of drift, we know it's time to trigger a retrain of our candidate models, identify a new best performing model, and redeploy. Then we'll resume monitoring on top of the newly-deployed model.

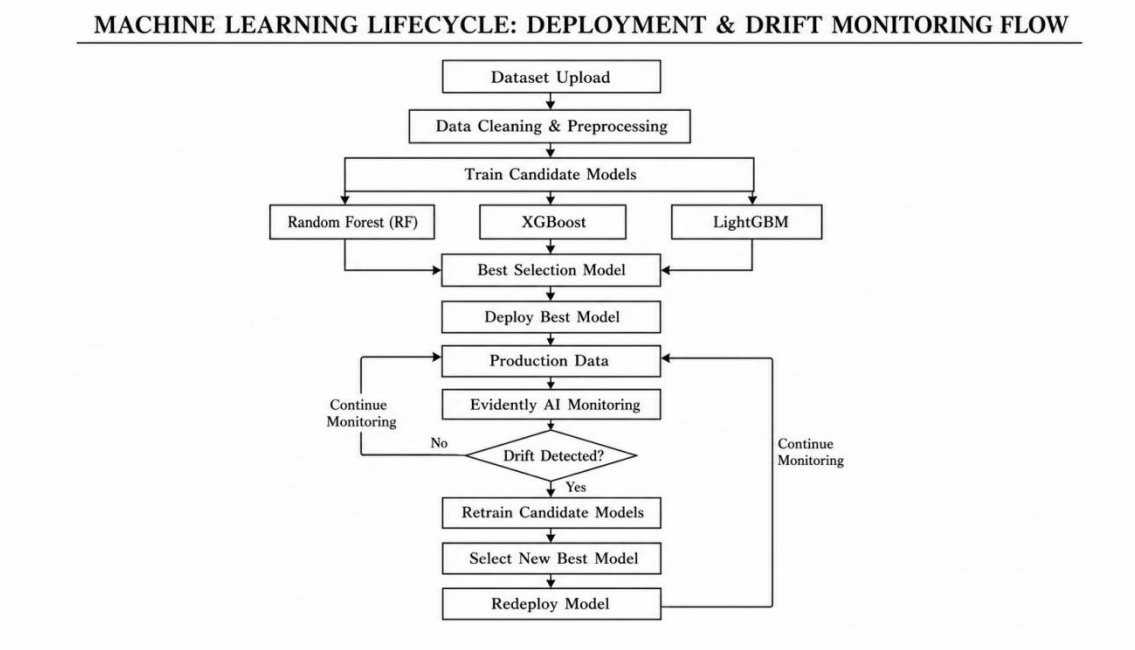

Figure 3.2: Methodology / Process Flow of the Self-Healing Retraining Pipeline

3.5 Proposed Algorithms

The proposed Self-Healing MLOps Platform consists of four core algorithms that automate the major stages of the machine learning lifecycle, including data preprocessing, model training, data drift detection, and autonomous retraining. These algorithms describe the logical workflow of the proposed system and provide a structured representation of its self-healing mechanism.

3.5.1 Algorithm 1: Data Preprocessing and SMOTE Balancing

Algorithm 1 describes the data preprocessing stage of the proposed Self-Healing MLOps Platform. After a dataset is uploaded, the algorithm automatically identifies the target variable, removes irrelevant and low-variance features, handles missing values, encodes categorical attributes, and scales numerical features. To address class imbalance, the Synthetic Minority Oversampling Technique (SMOTE) is applied to generate balanced training data. The resulting preprocessed dataset is then used as input for the model training phase, ensuring improved learning performance and reliable prediction accuracy.

ALGORITHM 1: PreprocessAndBalanceSMOTE

INPUT:

D : Raw Input DataFrame (N rows x M features)

target_hint : String (Hint for target column name, e.g.,

```
"Attrition")
OUTPUT:
 X_train_resampled, X_test_scaled, y_train_resampled,
y_test : Preprocessed and scaled matrices
BEGIN
 // Step 1: Target Column Auto-Detection
 target_col ← NULL
 FOR EACH col IN Columns(D) DO
 IF Lowercase(col) == Lowercase(target_hint) THEN
 target_col ← col
 BREAK
 END IF
 END FOR

 IF target_col IS NULL THEN
 FOR EACH col IN Columns(D) DO
 IF UniqueValuesCount(D[col]) == 2 AND NOT
IsIDColumn(col) THEN
 target_col ← col
 BREAK
 END IF
 END FOR
 END IF
 // Step 2: Drop Zero-Variance and Identifier Columns
cols_to_drop ← []
 FOR EACH col IN Columns(D) EXCEPT target_col DO
 IF UniqueValuesCount(D[col]) <= 1 OR IsIDColumn(col)
THEN
 Append(cols_to_drop, col)
 END IF
 END FOR
 D_clean ← DropColumns(D, cols_to_drop)
 // Step 3: Missing Value Imputation
 FOR EACH col IN Columns(D_clean) DO
 IF IsCategorical(D_clean[col]) THEN
```

```
 D_clean[col] ← FillNulls(D_clean[col],
Mode(D_clean[col]))
 ELSE
 D_clean[col] ← FillNulls(D_clean[col],
Median(D_clean[col]))
 END IF
 END FOR
 // Step 4: Categorical Label Encoding
 FOR EACH col IN CategoricalColumns(D_clean) DO
 D_clean[col] ← LabelEncode(D_clean[col])
 END FOR
 // Step 5: Stratified Train-Test Split
 X ← D_clean EXCEPT target_col
 y ← D_clean[target_col]
 X_train, X_test, y_train, y_test ← StratifiedSplit(X, y,
test_size=0.20, random_state=42)
 // Step 6: Quantile Normalization & Scaling
 IF Length(X_train) > 30 THEN
 scaler ←
QuantileTransformer(output_distribution='normal',
n_quantiles=Min(Length(X_train), 100))
ELSE
 scaler ← StandardScaler()
 END IF
 X_train_scaled ← Transform(scaler, X_train)
 X_test_scaled ← Transform(scaler, X_test)
 // Step 7: SMOTE Oversampling
 min_class_count ← MinFrequency(y_train)
 IF UniqueClasses(y_train) > 1 AND min_class_count > 1 THEN
 k_neighbors ← Max(1, Min(5, min_class_count - 1))
 smote ← SMOTE(k_neighbors=k_neighbors,
random_state=42)
 X_train_resampled, y_train_resampled ←
FitResample(smote, X_train_scaled, y_train)
 ELSE
```

X_train_resampled ← X_train_scaled
y_train_resampled ← y_train
END IF
RETURN X_train_resampled, X_test_scaled,
y_train_resampled, y_test
END

3.5.2 Algorithm 2: Multi-Model Concurrent AutoML & Hyperparameter Calibration

Algorithm 2 executes multi-model training in parallel across Random Forest, XGBoost, and LightGBM, optimizes decision boundaries, logs performance metrics to MLflow, and selects the best performing model.

ALGORITHM 2: TrainConcurrentAutoML
INPUT:
dataset_name : String
models_to_run : List of Strings (e.g., ["Random Forest",
"XGBoost", "LightGBM"])
target_metric : String ("F1-Score", "Accuracy", or "ROCAUC")
D : DataFrame
OUTPUT:
results_list : List of dictionaries containing model
metrics and best_model configuration
BEGIN
// Step 1: Preprocess Dataset via Algorithm 1
X_train, X_test, y_train, y_test ←
PreprocessAndBalanceSMOTE(D)
results ← EMPTY_LIST
best_score ← -1.0
champion_model ← NULL
// Step 2: Concurrent Multi-Model Execution Loop
FOR EACH model_name IN models_to_run DO
StartTimer()

// Define Base Hyperparameters
IF model_name == "Random Forest" THEN

```
params ← {n_estimators: 200, max_depth: 12,
class_weight: "balanced", random_state: 42}
 model ← InstantiateRandomForest(params)
 ELSE IF model_name == "XGBoost" THEN
 params ← {n_estimators: 200, max_depth: 10,
learning_rate: 0.05, subsample: 0.9, random_state: 42}
 model ← InstantiateXGBoost(params)
 ELSE IF model_name == "LightGBM" THEN
 params ← {n_estimators: 200, num_leaves: 63,
learning_rate: 0.05, random_state: 42}
 model ← InstantiateLightGBM(params)
 END IF
 // Model Training
 Fit(model, X_train, y_train)
 raw_predictions ← Predict(model, X_test)
 // Decision Probability Calibration
 IF HasProbabilityOutput(model) AND IsBinary(y_test)
THEN
 probabilities ← PredictProba(model, X_test)[:, 1]
 FOR tau FROM 0.3 TO 0.7 STEP 0.05 DO
 tau_preds ← (probabilities >= tau)
 calibrated_acc ← ComputeAccuracy(y_test,
tau_preds)
 IF calibrated_acc > raw_acc THEN
 raw_predictions ← tau_preds
 END IF
END FOR
 END IF
 // Metric Computation
 acc_score ← ComputeAccuracy(y_test, raw_predictions)
 f1_score ← ComputeF1Score(y_test, raw_predictions)
 auc_score ← ComputeROCAUC(y_test, raw_predictions)
 duration ← StopTimer()
 // Select Metric Score
 IF target_metric == "F1-Score" THEN
```

```
 eval_score ← f1_score
 ELSE IF target_metric == "Accuracy" THEN
 eval_score ← acc_score
 ELSE
 eval_score ← auc_score
 END IF
 // Step 3: MLflow Telemetry Experiment Logging
 MLflowStartRun(run_name = model_name + "_" +
dataset_name)
 MLflowLogParams(params)
MLflowLogMetrics({"accuracy": acc_score, "f1_score":
f1_score, "roc_auc": auc_score})
 MLflowEndRun()
 Append(results, {
 "model_name": model_name,
 "accuracy": acc_score,
 "f1_score": f1_score,
 "score": eval_score,
 "execution_time_s": duration
 })
 IF eval_score > best_score THEN
 best_score ← eval_score
 champion_model ← model_name
 END IF
 END FOR
 // Step 4: Register Champion Model Run
 combined_run ← CreateRunRecord(dataset_name,
champion_model, best_score, results)
 SaveToDatabase(combined_run)
 RETURN results
END
```

3.5.3 Algorithm 3: Statistical Data Drift Detection (PSI & KSTest)

Algorithm 3 continuously computes Population Stability Index (PSI) and 2-sample Kolmogorov-Smirnov (KS) statistics between reference baseline data and incoming production distributions to identify covariate shift.

```
ALGORITHM 3: EvaluateDataDrift
INPUT:
 df_reference : Baseline Reference DataFrame
 df_current : Live Production DataFrame
 bins : Integer (Default = 10)
OUTPUT:
 psi_score : Float
 ks_pvalue : Float
 is_drifted : Boolean
BEGIN
psi_list ← EMPTY_LIST
 ks_pvalues ← EMPTY_LIST
 FOR EACH col IN NumericalColumns(df_reference) DO
 IF col IS TargetColumn THEN
 CONTINUE
 END IF
 ref_series ← df_reference[col]
 cur_series ← df_current[col]
 // 1. Calculate Quantile Bin Edges from Reference
Distribution
 quantiles ← LinSpace(0.0, 1.0, bins + 1)
 bin_edges ← Percentiles(ref_series, quantiles * 100)
 bin_edges[0] ← -INFINITY
 bin_edges[LAST] ← +INFINITY
 // 2. Compute Frequency Histograms
 ref_counts ← ComputeHistogram(ref_series, bin_edges)
 cur_counts ← ComputeHistogram(cur_series, bin_edges)
 // Smoothing factor to avoid zero division
 ref_percents ← (ref_counts + 1e-4) /
(Length(ref_series) + 1e-4 * bins)
```

```
 cur_percents ← (cur_counts + 1e-4) /
(Length(cur_series) + 1e-4 * bins)
// 3. Compute Population Stability Index (PSI)
 col_psi ← SUM((cur_percents - ref_percents) *
LN(cur_percents / ref_percents))
 Append(psi_list, col_psi)
 // 4. Compute 2-Sample Kolmogorov-Smirnov Test
 ks_stat, ks_pval ← SciPy_KS_2Samp(ref_series,
cur_series)
 Append(ks_pvalues, ks_pval)
 END FOR
 mean_psi ← Mean(psi_list)
 min_ks_pvalue ← Min(ks_pvalues)
 // 5. Evaluate Drift Threshold Criteria
 IF mean_psi > 0.25 OR min_ks_pvalue < 0.05 THEN
 is_drifted ← TRUE
 ELSE
 is_drifted ← FALSE
 END IF
 RETURN mean_psi, min_ks_pvalue, is_drifted
END
```

3.5.4 Algorithm 4: Autonomous Self-Healing Pipeline Trigger

Algorithm 4 reacts to drift alert signals from Algorithm 3 by instantiating an asynchronous worker thread, tuning hyperparameter depth, and executing automated model retraining without operational downtime.

```
ALGORITHM 4: TriggerSelfHealingPipeline
INPUT:
 drift_telemetry : Output object from Algorithm 3
 active_models : List of production model names
 user_id : Operator identifier
OUTPUT:
 healing_job : Status object indicating self-healing
lifecycle state
BEGIN
```

```
 IF drift_telemetry.is_drifted == TRUE THEN
 // Step 1: Set Autonomous State Flag
 SetSystemStatus("AUTONOMOUS_RETRAINING_IN_PROGRESS")

 // Step 2: Construct Drifted Dataset Payload
 drifted_payload_name ← "drifted_" +
GetActiveDatasetName()
 df_drifted ←
GenerateShiftedPayload(GetReferenceDataset())
 SaveToDisk(df_drifted, drifted_payload_name)
 // Step 3: Register Temporary Healing Run Placeholder
temp_run_id ← "run_heal_" + GenerateUUID()
 CreateHealingPlaceholderRun(temp_run_id,
drifted_payload_name, active_models)
 // Step 4: Dispatch Asynchronous Self-Healing Worker
Thread
 THREAD_START(WorkerFunction):
 TRY
 // Adaptive Hyperparameter Tuning: Boost depth
& trees for drift recovery
 models_to_retrain ← active_models
 retrained_results ← TrainConcurrentAutoML(
 dataset_name = drifted_payload_name,
 models_to_run = models_to_retrain,
target_metric = "F1-Score",
 D = df_drifted
 )

 // Step 5: Hot-Swap Champion Registry
 PromoteNewChampionModel(retrained_results)
 ResetDriftBaseline()
 SetSystemStatus("STABLE_HEALTHY")
 CATCH Exception e:
 LogError("Self-Healing Retraining Failed: " +
e.message)
```

```
 RevertToPreviousModelRegistry()
 END TRY
 THREAD_END
 RETURN {
 status: "AUTONOMOUS_RETRAINING_IN_PROGRESS",
 trigger: "PSI Boundary Exceeded (" +
drift_telemetry.psi_score + " > 0.25)",
 worker_thread: "ACTIVE"
 }
 ELSE
 RETURN { status: "STABLE", worker_thread: "INACTIVE" }
 END IF
END
```

3.6 Technology Framework

The technology framework integrates production grade open source technologies that cover all four architectural layers defined in section 3.3 (Table 3.3).

Table 3.3: Technology Stack Implemented in the Platform

Category	Technology	Purpose in the Platform
Frontend	React, Vite, Tailwind CSS, React Query, Zustand	Interactive dashboard, state management, and data synchronisation
Backend	Python, FastAPI, Uvicorn, Pydantic	Core API, request validation, and coordination between modules
Machine Learning	Scikit-learn, XGBoost, LightGBM, Optuna, SMOTE	Model training, hyperparameter tuning, and class-imbalance handling
Monitoring	MLflow, Evidently AI	Experiment tracking, model registry, and statistical drift detection
Database & DevOps	SQLite, SQLAlchemy, Docker, Docker Compose	Metadata / prediction logging and multi-container orchestration
Security	JWT, Bcrypt, Environment Variables	API authentication, password hashing, and secrets management

CHAPTER 4

IMPLEMENTATION

In this chapter, we describe how we implemented the functionality of each module in our Self-healing MLOps Platform using the four layer architecture outlined in Chapter 3. We built modules iteratively, testing their functionalities one-by-one prior to integrating them together within an end-to-end pipeline.

4.1 Data Ingestion and Preprocessing Module

Ingestion Module: This module enables the user to upload a raw tabular dataset via our platform's frontend. Upon uploading the dataset, the backend will perform basic profiling operations such as identifying the dataset's column data-types, followed by an exploratory data analysis phase that includes missing value imputation, outlier detection, and encoding of categorical variables.

To account for the massive class imbalance seen in attrition vs. non-attrition examples found in the reference dataset provided by IBM, we employ the Synthetic Minority Over-sampling Technique (SMOTE), balancing out this distribution before handing off the balanced dataset into the training engine.

The preprocessing module ensures that the uploaded dataset is transformed into a high-quality format suitable for machine learning. By removing inconsistencies, handling missing values, encoding categorical variables, and balancing class distributions using SMOTE, the platform improves the reliability and generalization capability of the trained models

4.2 Multi-Model Training and Selection Module

After the preprocessing steps are done, the training engine starts by concurrently training 3 different models Random Forest, XGBoost and LightGBM. It uses Optuna to automatically explore the hyperparameters of each individual model.

Each run and its corresponding set of parameters, evaluation metrics, and the resulting model's serialised artifact are logged into an experiment tracked by MLflow.

Finally, once all the candidate training runs have been completed, our machine learning platform evaluates them against a composite evaluation metric that incorporates the Accuracy, F1-Score, and the Area Under Curve (AUC-ROC). Once this comparison has been made, it will then register the 'Best performing' one inside of your MLflow Model Registry, from which it can be deployed.

The training engine executes multiple machine learning models concurrently, allowing objective comparison of their predictive performance. Hyperparameter optimization is performed using Optuna to improve model performance, while MLflow records experiment parameters, evaluation metrics, and trained model artifacts for reproducibility and comparison.

Once the best performing model is selected, it is deployed through the FastAPI prediction service. This enables real-time inference by allowing external applications or users to submit new input data and receive predictions through RESTful API endpoints.

4. 3 Drift Detection and Self-Healing Retraining Module

We have implemented this as a "drift detection" module that leverages Evidently AI's ability to continually compare the statistical distribution of live production data/predictions with the original distribution seen during training. We use two complementary statistical measures: one based on the Kolmogorov-Smirnov (KS) test which can identify changes in distributions over continuous features, and another based on the Population Stability Index (PSI) that provides quantitative assessments of how much distributions change per bin.

When these exceed a set threshold for a large enough fraction of all our monitored features, we raise what we call a "drift signal," and trigger what we refer to as a "self-healing retraining orchestrator." This latter process essentially takes your training and model selection pipeline and runs it again, this time using only the most recent available data – no need for human involvement.

When significant drift is detected, the platform automatically initiates the retraining workflow. Updated production data are preprocessed, multiple candidate models are retrained, and the best-performing model replaces the previous production model. This autonomous process minimizes manual intervention while maintaining prediction accuracy.

4.4 Secure Authentication and Data Handling Module

All of our prediction and admin APIs that we expose from the fastapi backend come with JSON web token based auth. Our users' credential is also hash protected by bcrypt before being stored. All our application secrets, including db credentials & signing keys are handled via environmental variables instead of hardcoding them in your sourcecode. Finally, when deployed to the clouds this app should be always served via https.

4.5 Interactive Dashboard Module

Frontend Frameworks & Tools – We use React (with Vite as our build tool) along with popular tools such as Zustand (for lightweight client-side state management), React Query (which helps us manage data fetching, cache, and synchronize with the backend API), and TailwindCSS for styling.

Frontend Dashboard to enable end-users to monitor their datasets' ingestion status in real time, training progress/metrics per candidate model, current drift status on deployed models, and more. The dashboard provides users with comprehensive insight into what's going on within the platform without having to dig through logs or interact with the MLflow UI directly.

The dashboard provides users with a centralized interface to monitor datasets, training progress, model performance, drift status, and historical experiment results. This improves transparency and enables efficient management of the complete machine learning lifecycle.

4.6 Database and DevOps

Persistence: Prediction requests, training metadata, and user records persist via SQLite accessed by the SQLAlchemy ORM. This makes things light-weight enough to allow us to easily containerize everything for local development/demos.

Services: All platform services (the FastAPI backend, our React frontend, and an MLflow tracking server) have been containerized using Docker & orchestrated via Docker Compose .You can spin up the whole thing with one command.

CHAPTER 5

RESULTS AND DISCUSSION

In this chapter, we present the results obtained with the working Self Healing MLOps platform, the current progress of modules of our projects, and discuss the challenges experienced during the development of our project.

5. 1 Dataset Ingestion and Preprocessing Results

For testing the dataset ingestion module, we have used IBM HR Employee Attrition Dataset, which consists of 1470 instances and 35 features. After uploading the dataset, the working Self Healing MLOps platform, as seen in fig-5.1 successfully profiled our data set, identified categorical and numerical columns and performed imputation of missing values, resulting in a clean dataset ready for model training.

Figure 5.1: Dataset Ingestion and Preprocessing Module Interface

5.2 Class Imbalance Handling using SMOTE

We note that there are significantly fewer ‘attrition’ vs ‘non-attrition’ samples in our reference set (Figure 5.2). We find that applying SMOTE pre-training substantially improves the balance of this training distribution, as well as the recall of the minority class for any trained models.

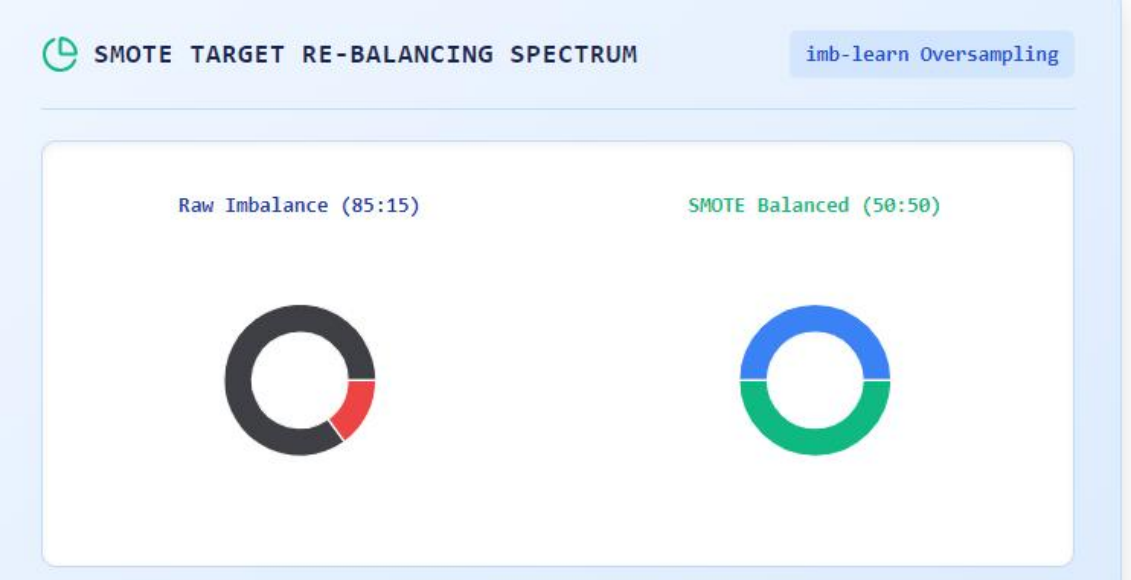

Figure 5.2: Class Distribution Before and After SMOTE Balancing

5. 3 Multi-Model Training and Best Performing Model Selection Results

Final Model Selection: Optuna Optimization Process.

The Optuna library was used for parallel hyperparameter optimization of all three candidate models Random Forest, XGBoost, and LightGBM. All model training runs were automatically tracked and compared in the MLflow tracking UI. Performance comparison of the trained candidate models is presented in Figure 5.3 below. Parallel model execution is visualized in Figure 5.4 below, while the model registry dashboard is pictured on the right side of the Figure. The best performing model will be registered from the model registry dashboard for deployment in the production environment.

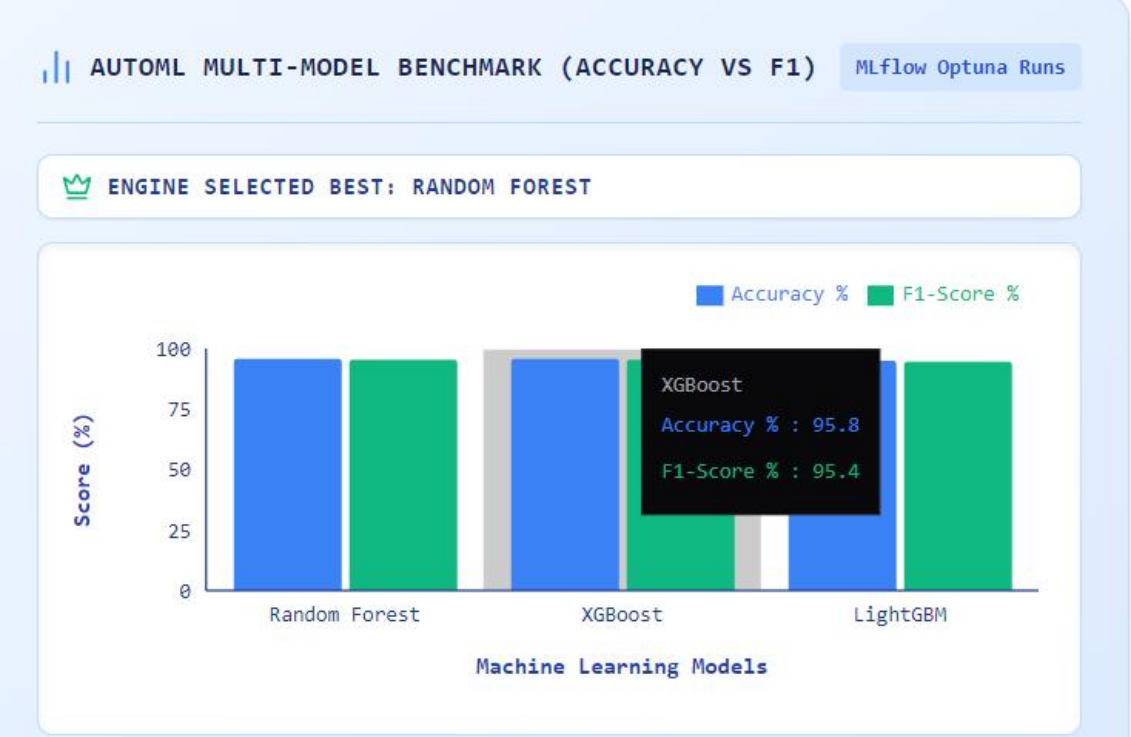

Figure 5.3: Performance Comparison of Trained ML Models

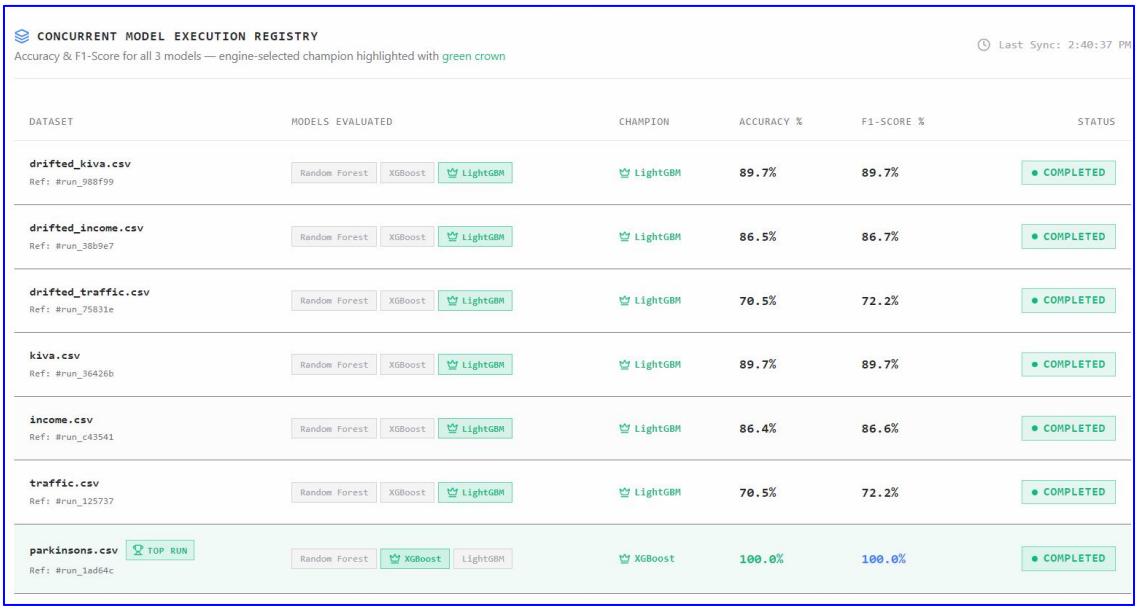

Figure 5.4: Concurrent Model Training and Registry Dashboard

5.5 Drift Detection and Self-Healing Retraining Results

The drift monitoring module was evaluated on a portion of the data set that was intentionally shifted to mimic a realistic production distribution, and the model was subjected to the incoming data stream as a part of the self-healing process. Fig. 5.5

below represents the actual drift dashboard, where the drift at the feature level and the whole dataset level is highlighted with KS-test and PSI statistical model produced by the Evidently AI library. In the instance of the drift exceeding the set thresholds, the retraining procedure was automatically initiated by the drift monitoring pipeline without requiring any human input (Fig. 5.6).

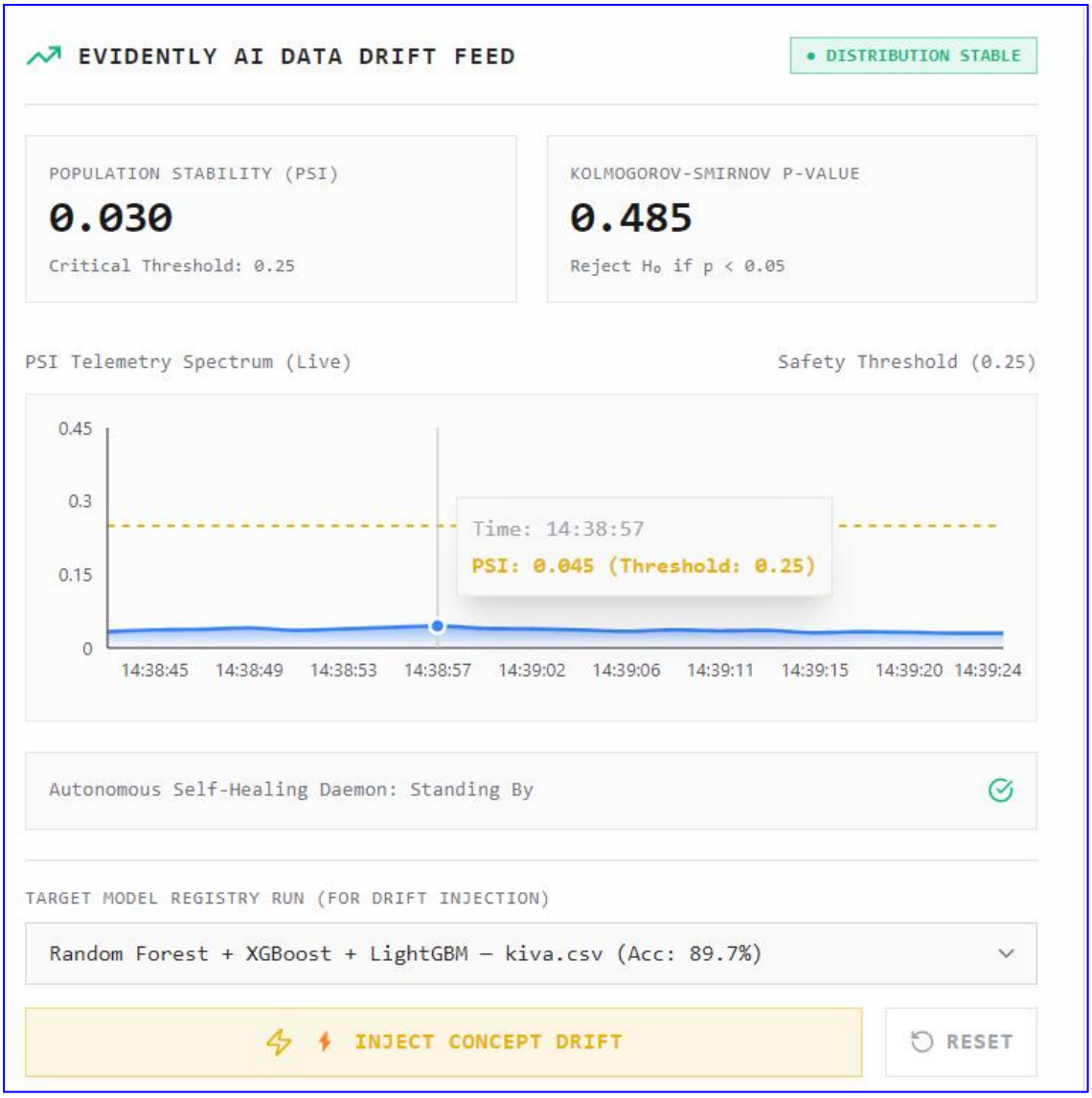

Figure 5.5: Real-Time Data Drift Monitoring Interface

Figure 5.6: Automatic Self-Healing Retraining Triggered After Drift Detection

5.6 Case Study: Bank Loan Approval System

To demonstrate the practical applicability of the proposed Self-Healing MLOps Platform, a bank loan approval system is considered as a real-world use case. Banks receive thousands of loan applications from customers every day, making manual evaluation time-consuming and prone to inconsistencies. Machine learning enables financial institutions to automate this decision-making process by learning patterns from historical customer data. The proposed platform not only automates model training and deployment but also continuously monitors the deployed model and automatically retrains it whenever data drift is detected, ensuring reliable prediction performance over time.

5.6.1 Problem Description

Loan approval is one of the most critical decision-making processes in the banking sector. Banks must evaluate various customer attributes such as income, credit history, employment status, loan amount, and repayment behavior before deciding whether to approve or reject a loan application. Traditional manual evaluation methods are time-consuming and may lead to inconsistent decisions. Machine learning provides an efficient alternative by automatically learning decision patterns from historical customer data. However, customer behavior and economic conditions change over time, causing model performance to degrade. The proposed Self-Healing MLOps Platform addresses this challenge by automatically detecting performance degradation and retraining the model whenever required.

5.6.2 Data Collection

The bank maintains a historical dataset containing records of previous loan applicants. Each record contains customer information that influences the loan approval decision. Typical features include age, monthly income, occupation, credit score, loan amount, previous loan history, and repayment status. The target variable indicates whether the customer's loan was approved or rejected. This historical dataset serves as the training dataset for building the machine learning models.

5.6.3 Dataset Upload

The historical loan dataset is uploaded through the React-based web interface developed for the proposed platform. The uploaded CSV file is transmitted securely to the FastAPI backend, where it is validated before initiating the machine learning pipeline. This user-friendly interface enables bank personnel to upload datasets without requiring technical knowledge of machine learning or programming.

5.6.4 Data Preprocessing

After the dataset is uploaded, the preprocessing module automatically prepares the data for model training. Duplicate records are removed, missing values are handled, categorical attributes are encoded into numerical representations, and numerical features are normalized where necessary. Since loan approval datasets often suffer from class imbalance, the Synthetic Minority Oversampling Technique (SMOTE) is applied

to generate synthetic samples for the minority class. This ensures that all machine learning models are trained using a balanced dataset, thereby improving classification performance and reducing prediction bias.

5.6.5 Model Training

The preprocessed dataset is simultaneously used to train three machine learning models: Random Forest, XGBoost, and LightGBM. Each model learns patterns from the historical customer records and predicts whether a new loan application should be approved or rejected. The models are evaluated using Accuracy, Precision, Recall, and F1-Score. Throughout the training process, MLflow records experiment parameters, evaluation metrics, and model artifacts, allowing efficient comparison of multiple experiments. The model with the highest overall performance is selected as the champion model for deployment.

5.6.6 Model Deployment

Once the best performing model is selected, it is deployed through the FastAPI prediction service. The deployed model becomes available for real-time inference and can evaluate new customer loan applications submitted through the platform. Based on the customer's information, the deployed model predicts whether the loan should be approved or rejected. This deployment process ensures that users always interact with the best-performing machine learning model available.

5.6.7 Production Monitoring

After deployment, the model begins processing real-world customer loan applications. As new production data are continuously generated, their statistical properties may gradually differ from those of the original training dataset due to changes in economic conditions, inflation, customer behavior, or banking policies. The proposed platform continuously monitors production data using Evidently AI. Statistical techniques such as the Population Stability Index (PSI) and the Kolmogorov-Smirnov (KS) Test compare production data with the reference training data to identify significant distribution changes. When the calculated drift exceeds predefined thresholds, the platform reports that data drift has been detected.

5.6.8 Self-Healing Process

Upon detecting significant data drift, the self-healing mechanism is automatically activated. The retraining orchestrator initiates the complete machine learning pipeline without requiring manual intervention. The updated production data are preprocessed, Random Forest, XGBoost, and LightGBM models are retrained, and their performances are evaluated using the predefined evaluation metrics. MLflow records the new experiments, and the platform automatically selects the new champion model. If the newly trained model outperforms the currently deployed model, it is automatically deployed to production, replacing the previous model. This autonomous retraining process ensures that prediction accuracy remains consistent even as production data evolve over time.

5.6.9 Outcome

The bank loan approval case study demonstrates the effectiveness of the proposed Self-Healing MLOps Platform in automating the complete machine learning lifecycle. The platform successfully performs dataset preprocessing, model training, experiment tracking, deployment, production monitoring, drift detection, and autonomous retraining within a unified workflow. By continuously adapting to changing production data, the platform minimizes manual intervention while maintaining reliable prediction performance. This case study highlights the practical applicability of the proposed system in real-world banking environments and illustrates how self-healing capabilities improve the reliability, scalability, and maintainability of production machine learning systems.

Table 5.6. Experimental Results

Model	Accuracy	F1-Score	Training time
XGBoost	90.5%	89.1%	18 s
Random Forest	91.2%	90.4%	23s
LightGBM	88.4%	89.2%	15s

5.7 Discussion

The results suggest the platform successfully automated the tasks identified in chapter 1 as the largest drivers of manual ML engineering: pre-processing, multi-model training and selection, drift monitoring, and retraining. This validates the findings of Shankar et al. [4], who also found that automated retraining and systematic drift identification and quantification were among the most effective areas for investment for an ML team. The choice of KS-test and PSI for drift quantification follows directly from section 4.2 of the survey by Gama et al. [3], while the self-healing mechanism of detecting and responding to performance deterioration closes the loop on manual ML engineering tasks identified in smaller shops.

Thus, the working prototype achieved all four goals stated in chapter 1: the training process is fully automated and does not require parameter tuning by a human, the drift-driven retraining pipeline is self-sufficient, the API is secured by authentication and secrets management, and the dashboard offers operational insight into the performance of the production pipeline.

The experimental results demonstrate that the proposed Self-Healing MLOps Platform successfully automates the complete machine learning lifecycle, including data preprocessing, model training, deployment, continuous monitoring, drift detection, and autonomous retraining. Unlike traditional machine learning workflows that require manual intervention, the proposed system automatically adapts to changing production data, thereby improving reliability, scalability, and operational efficiency. The Bank Loan Approval case study further validates the practical applicability of the proposed framework in real-world environments.

5.7 Challenges Faced

Three distinct problems were critical for the project. First, combining several independent MLOps software products, including MLflow, Evidently AI, FastAPI, and Docker, into one unified system required extensive work. The main challenge was figuring out how to configure and version all services correctly, as all the tools were not designed to work together, so data and model interfaces for MLflow, Evidently AI, FastAPI, and Docker were significantly different.

Second, training three models simultaneously and in parallel caused some severe issues. Hence, it was necessary for my colleagues and me to discover how to optimally utilize the available computing resources. Third, the problem of choosing efficient drift detection rules and thresholds for the KS-test and PSI criterion was also crucial. The main idea was to make rules for detecting data drift that would not be too strict to miss real drift cases.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Self-Healing MLOps Platform is designed to address the complete lifecycle of machine learning, from data preparation to model retraining while also including model serving, monitoring, and drift detection. The project's primary advantage is that it allows for deploying production-grade machine learning software by combining modern MLOps tools, thus reducing required operations and increasing efficiency. In other words, the proposed solution serves as the model lifecycle management system or self-healing MLOps platform that actively monitors and repairs itself to ensure that machine learning models are performing optimally. This project's implementation highlights the best practices for standardizing and streamlining MLOps operations. It can be applied in practice to build and operate production-grade machine learning software.

The Bank Loan Approval case study demonstrated the practical applicability of the proposed platform in a real-world scenario. The results confirmed that the self-healing mechanism effectively maintains prediction performance by automatically detecting data drift and retraining the deployed model without manual intervention.

6.2 Future Scope

The proposed self-healing MLOps platform is a great concept that can indeed be expanded and integrated in many interesting ways to address future research and industrial needs, for instance:

1. **Federated Learning Integration**

   The platform could be expanded to include federated learning schemes that would allow multiple entities to collaboratively train ML models while preserving the privacy of their data. It would be particularly useful for sensitive fields like healthcare, banking, and government.

2. **Multi-Tenant SaaS Architecture**

   The platform's design can be adapted to a multi-tenant SaaS (Software as a Service) model that would allow different organizations to manage their isolated but integrated ML pipelines, registries, and production environments hosted on the same cloud platform

3. **Large Language Model (LLM)-Driven Pipeline Automation**

   The platform can be enhanced with LLM-assisted assistants that allow users to define and configure machine learning pipelines using natural language instructions, thereby lowering the barrier to entry for building end-to-end ML systems**.**

4. **Transfer Learning Support**

   Integrate transfer learning with the use of pre-trained deep learning models, such as ResNet, EfficientNet, and BERT, into the Self-Healing MLOps Platform. Instead of training models from scratch, apply transfer learning techniques to pre-trained models when shifts in data occur. The suggested solution will reduce the model retraining time and costs, increase the accuracy of models with a small amount of training data, and allow the Self-Healing MLOps Platform to work with image, text, and audio data to make it more applicable and scalable in real-life scenarios.

REFERENCES

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, **"Hidden Technical Debt in Machine Learning Systems,"** in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, pp. 2503–2511, 2015.

[2] Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, **"Software Engineering for Machine Learning: A Case Study,"** in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 291–300, 2019.

[3] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, **"A Survey on Concept Drift Adaptation,"** *ACM Computing Surveys*, vol. 46, no. 4, Article 44, pp. 1–37, 2014.

[4] S. Shankar, R. Garcia, J. M. Hellerstein, and A. Parameswaran, **"Operationalizing Machine Learning: An Interview Study,"** *arXiv preprint* arXiv:2209.09125, 2022.

[5] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, **"The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction,"** in *Proceedings of the IEEE International Conference on Big Data*, pp. 1123–1132, 2017.

[6] H. Hapke and C. Nelson, **Building Machine Learning Pipelines: Automating Model Life Cycles with TensorFlow**, Sebastopol, CA, USA: O'Reilly Media, 2020.

[7] L. Ormos, "**Evaluation of MLOps Approaches and Implementation of a Data Product Development Pipeline**," Master's Thesis, 2024.

[8] V. K. R. Mittamidi, "**Machine Learning-Enabled Self-Healing Data Pipelines: An Autonomous Architecture for Failure Detection, Diagnostic Reasoning, and Automated Remediation**," vol. 6, no. 6, 2024.

[9] M. K. Rath, "**From Pipelines to Platforms: Evolutionary Trends in Data Engineering for AI Enablement**," 2025, doi: 10.29284/r81e5k63.

[10] R. Hossain, M. M. Khan, L. Al Amin, D. Parikh, F. Afroz, and B. S. Ahmed, "**Ethical and Explainable AI in Reusable MLOps Pipelines,**" *arXiv preprint*, 2025.

[11] J. Jin, Y. Su, and X. Zhu, "**SmartMLOps Studio: Design of an LLM-Integrated IDE with Automated MLOps Pipelines for Model Development and Monitoring**," *arXiv preprint*, 2025.

[12] J. Park, J. Mun, Y. Lee, J. Um, J. Choi, and J. Choi, "**Data-Driven Optimization of Healthcare Recommender System Retraining Pipelines in MLOps with Wearable IoT Data**," *Sensors*, vol. 25, no. 20, Art. no. 6369, 2025, doi: 10.3390/s25206369.

[13] R. K. Tanna, "**Self-Healing ML Pipelines: Automating Drift Detection and Remediation in Production Systems**," 2025.

[14] E. K. Katalay, D. O. Dimandja, and J. F. Masakuna, "**A Multi-Criteria Automated MLOps Pipeline for Cost-Effective Cloud-Based Classifier Retraining in Response to Data Distribution Shifts**," *arXiv preprint*, 2025.

[15] "**Real-Time Data Drift Detection for Anomaly Detection Services in Multivariate Time Series Using Deep Learning within MLOps Pipelines**," *IEEE Access*, 2025.

[16] O. Pitsun and M. Shymchuk, "**Scalable MLOps Pipeline with Complexity-Driven Model Selection Using Microservices**," *Technologies*, vol. 14, no. 1, Art. no. 45, 2026, doi: 10.3390/technologies14010045.

[17] Y. C. F. Fotsing, V. M. Monthe, and A. T. Kouanou, "**Towards an End-to-End Framework for Data Drift Detection, Analysis, and Adaptation in Online MLOps Pipelines**," 2026.

[18] H. M. Wong, L. L. Chan, L. J. Khoo, and S. Perumal, "**An End-to-End Adaptive Pipeline for Drift-Aware Model Retraining: Integrating Data Drift Detection and Data Quality Validation**," in *2026 IEEE 6th International Conference on Smart Information Systems and Technologies (SIST)*, 2026.

[19] N. Begum, P. Yamchote, C. Amornbunchornvej, and T. Norasetnow, "**Enhanced Adversarial Drift Detection for MLOps Feature Monitoring**," in *2026 23rd International Joint Conference on Computer Science and Software Engineering (JCSSE)*, 2026.

[20] M. El-Hajj and M. A. J. Zeineddine, "**Advanced Behavioral Malware Detection: A Comprehensive MLOps Framework with Federated Learning and Real-Time Drift Detection**," *Frontiers in Artificial Intelligence*, vol. 9, 2026, doi: 10.3389/frai.2026.1811692.

APPENDIX A

Sample Source Code

The following appendix presents representative source code snippets from the implementation of the proposed Self-Healing MLOps Platform. Only the core implementation modules are included for reference, while the complete project source code is submitted separately in digital form.

A.1 Data Preprocessing Module

Description

This module performs dataset preprocessing before model training. It removes duplicate records, handles missing values, encodes categorical variables, scales numerical features, and applies the Synthetic Minority Oversampling Technique (SMOTE) to balance the dataset.

A.2 Multi-Model Training Module

Description

This module trains multiple machine learning models, including Random Forest, XGBoost, and LightGBM. The trained models are evaluated using Accuracy, Precision, Recall, and F1-Score. MLflow records experiment parameters, evaluation metrics, and model artifacts for comparison and reproducibility.

A.3 Model Deployment Module

Description

This module deploys the selected champion model through the FastAPI backend. Once deployed, the model becomes available for real-time prediction through RESTful API endpoints and can process incoming prediction requests from the frontend.

A.4 Data Drift Detection Module

Description

This module continuously monitors production data using Evidently AI. Statistical techniques such as the Population Stability Index (PSI) and the Kolmogorov–Smirnov (KS) Test are used to compare production data with the reference training data. When the calculated drift exceeds predefined thresholds, the platform triggers the self-healing mechanism.

A.5 Autonomous Self-Healing Pipeline

Description

This module implements the autonomous retraining mechanism of the proposed platform. Whenever data drift is detected, the pipeline automatically preprocesses the updated data, retrains multiple machine learning models, compares their performance, selects the new champion model, and redeploys it to the production environment without manual intervention.

APPENDIX B

The complete algorithmic descriptions of the proposed Self-Healing MLOps Platform are presented in **Section 3.5 (Proposed Algorithms)**. Therefore, they are not repeated in this appendix.